

1 Abstract

This report details the quality assurance analysis of high-frequency time-series chromatographic data, specifically focusing on the integrity of the UV_1.280_mAU signal. The primary objective was to quantify Signal-to-Noise (SNR) ratios, assess baseline stability, and detect data artifacts such as sensor drift and negative volume outliers that could compromise product safety and process reliability. The analysis involved a rigorous preprocessing workflow including baseline correction, missing data filtering, and outlier identification. Results indicate that while the UV_1.280_mAU signal exhibits high completeness (94%), significant intra-run noise variance and temporal drift were detected across the first five runs. Furthermore, specific runs were flagged as outliers due to excessive noise and the presence of negative volume_ml values exceeding -5 mL. These findings highlight critical data integrity issues requiring attention before downstream fraction collection and quality control decisions can be made.

2 Introduction

In the context of High-Performance Liquid Chromatography (HPLC), the accuracy of fraction collection and subsequent quality control decisions relies heavily on the stability and integrity of the chromatographic data. This analysis focuses on evaluating the quality of time-series data generated during a specific analytical run. The primary signal of interest, UV_1.280_mAU, was selected due to its superior data completeness (94%) compared to the UV_2.260_mAU signal, which suffered from significant missingness (70%). Critical process parameters, such as injection volume, were largely unavailable, necessitating a reliance on internal signal metrics for variability analysis. The main focus of this study was to identify excessive noise, sensor drift, and data artifacts—specifically negative baseline values and severe missingness—that could compromise the accuracy of the analysis. By quantifying the Signal-to-Noise (SNR) ratio and assessing baseline stability across available HPLC UV signals, this report aims to ensure product safety and process reliability.

3 Methodology

The data analysis workflow was structured into three primary phases: Data Integrity & Preprocessing, Noise Variance Analysis & Drift Detection, and Outlier Run Identification. First, the raw data from 'chromatography_combined.csv' underwent preprocessing. Rows where 'Fraction_unique_ID' was missing were filtered out to prevent skewing metrics. Baseline correction was applied to the 'UV_1.280_mAU' variable, while 'UV_2.260_mAU' was explicitly excluded from general quality metrics. Negative 'volume_ml' values less than -5 mL were flagged. Second, noise variance was calculated as the standard deviation of baseline segments for each 'run'. Temporal drift was assessed by plotting noise variance against 'Fraction_number' for the first five runs sorted by 'run_no'. The Signal-to-Noise (SNR) ratio was calculated using the median peak height versus baseline noise. Finally, runs were identified as outliers if their intra-run noise variance exceeded the mean plus two standard deviations. These flagged runs were then cross-referenced with counts of negative 'volume_ml' outliers. All outputs were generated to provide a comprehensive view of data quality, including summary statistics, visualizations of noise trends, and tables of anomalous runs.

4 Results and Discussion

The analysis revealed significant variations in data quality across the experimental runs. As illustrated in Figure 1, the top subplot displays the intra-run noise variance against the fraction number for the first five runs. The data indicates a clear temporal drift in noise levels, with variance increasing over time in several runs, suggesting sensor instability or baseline instability. The bottom subplot of Figure 1 presents a boxplot of the Signal-to-Noise (SNR) ratio grouped by run, highlighting substantial variability in signal quality. Runs with lower SNR values are associated with higher noise variance, directly impacting the reliability of peak detection and integration. Furthermore, the outlier identification phase flagged specific runs where the noise variance exceeded the statistical threshold ($\text{mean} + 2 \times \text{std}$). These flagged runs contained a notable number of negative 'volume_ml' values, indicating potential data acquisition errors or sensor saturation issues. The

exclusion of 'UV_2_260_mAU' was justified by its high missingness rate, ensuring that the quality metrics were derived from a robust dataset. The baseline correction applied to 'UV_1_280_mAU' effectively stabilized the signal for the initial analysis, yet the presence of negative outliers and drift suggests that further calibration or data filtering may be required for downstream applications.

5 Conclusion

This study successfully evaluated the integrity of the chromatographic data, confirming that while the UV_1_280_mAU signal is the most reliable source due to high completeness, it is susceptible to temporal drift and increased noise variance over time. The identification of outlier runs characterized by high noise and negative volume artifacts underscores the necessity of strict data quality controls before fraction collection. The methodology employed, including baseline correction and variance-based outlier detection, provided a robust framework for assessing data reliability. Future work should focus on mitigating the identified sensor drift and investigating the causes of negative volume readings to ensure the long-term accuracy and safety of the analytical process.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_dir = '/workspace/'

    # Load data
    df = pd.read_csv(input_path)

    # 1. Rename 'Unnamed: 0' to 'Fraction_number' if it exists
    if 'Unnamed: 0' in df.columns:
        df.rename(columns={'Unnamed: 0': 'Fraction_number'}, inplace=True)

    # 2. Filter rows where 'Fraction_unique_ID' is missing (case-insensitive check)
    filter_col = 'Fraction_unique_ID'
    # Normalize column names for robustness against naming variations
    df_lower = df.columns.str.lower()
    if filter_col.lower() in df_lower:
        actual_col = df.columns[df_lower == filter_col.lower()][0]
        df = df.dropna(subset=[actual_col])

    # 3. Flag negative 'volume_ml' values only if < -5 mL
    df['volume_flag'] = (df['volume_ml'] < -5)

    # 4. Store original UV values BEFORE baseline correction
    if 'UV_1_280_mAU' in df.columns:
        df['UV_1_280_mAU_original'] = df['UV_1_280_mAU'].copy()

    # 5. Apply baseline correction to 'UV_1_280_mAU'
```

```

if 'UV_1_280_mAU' in df.columns:
    median_uv = df['UV_1_280_mAU'].median()
    df['UV_1_280_mAU'] = df['UV_1_280_mAU'] - median_uv

# 6. Exclude 'UV_2_260_mAU'
if 'UV_2_260_mAU' in df.columns:
    df.drop(columns=['UV_2_260_mAU'], inplace=True)

# --- Output Generation ---

# 1. Count of filtered rows
filtered_count = len(df)

# 2. Summary stats before/after correction
# Ensure column exists before calculating median to avoid errors
uv_stats_before = {}
uv_stats_after = {}

if 'UV_1_280_mAU_original' in df.columns:
    uv_stats_before = {
        'min': df['UV_1_280_mAU_original'].min(),
        'max': df['UV_1_280_mAU_original'].max(),
        'mean': df['UV_1_280_mAU_original'].mean()
    }
    uv_stats_after = {
        'min': df['UV_1_280_mAU'].min(),
        'max': df['UV_1_280_mAU'].max(),
        'mean': df['UV_1_280_mAU'].mean()
    }

volume_stats_before = {
    'min': df['volume_ml'].min(),
    'max': df['volume_ml'].max(),
    'mean': df['volume_ml'].mean()
}

volume_stats_after = {
    'min': df['volume_ml'].min(),
    'max': df['volume_ml'].max(),
    'mean': df['volume_ml'].mean()
}

# 3. Top 20 worst negative outlier indices using 0(n) method
if 'UV_1_280_mAU' in df.columns:
    sorted_indices = df.nsmallest(20, 'UV_1_280_mAU').index.tolist()
else:
    sorted_indices = []

# Generate Markdown
md_content = f"""# Data Integrity & Preprocessing Report

## 1. Count of Filtered Rows
- **Total Rows After Filtering Missing Fraction_unique_ID**: {filtered_count}

## 2. Summary Statistics

### Before Baseline Correction
| Variable | Min | Max | Mean |
| :--- | :--- | :--- | :--- |

```

```

| volume_ml | {volume_stats_before['min']:.4f} | {volume_stats_before['max']:.4f}
| {volume_stats_before['mean']:.4f} |
| UV_1_280_mAU | {uv_stats_before['min']:.4f} | {uv_stats_before['max']:.4f} | {
uv_stats_before['mean']:.4f} |

### After Baseline Correction
| Variable | Min | Max | Mean |
| :--- | :--- | :--- | :--- |
| volume_ml | {volume_stats_after['min']:.4f} | {volume_stats_after['max']:.4f} |
{volume_stats_after['mean']:.4f} |
| UV_1_280_mAU | {uv_stats_after['min']:.4f} | {uv_stats_after['max']:.4f} | {
uv_stats_after['mean']:.4f} |

## 3. Top 20 Worst Negative Outlier Indices
| Rank | Index | UV_1_280_mAU (Corrected) |
| :--- | :--- | :--- |
"""

    for i, idx in enumerate(sorted_indices, 1):
        md_content += f"|_{i}|_{idx}|_{df.loc[idx, 'UV_1_280_mAU']:.4f}|_\\n"

    # Save to file
    output_file = os.path.join(output_dir, 'data_integrity_report.md')
    with open(output_file, 'w') as f:
        f.write(md_content)

    print(f"Report saved to {output_file}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns and data
    df = df[['run', 'Fraction_number', 'UV_1_280_mAU']].copy()
    df = df.sort_values(by=['run', 'Fraction_number'])

    # Identify unique runs
    unique_runs = df['run'].unique()

    # Select first 5 runs
    sorted_runs = sorted(unique_runs[:5])

    # Add Column Existence Check
    if 'run' not in df.columns or 'Fraction_number' not in df.columns or '
UV_1_280_mAU' not in df.columns:
        raise ValueError("Required columns not found in the input data.")

    # Calculate intra-run noise variance (std dev of baseline segments)
    # Using vectorized operations instead of iterating over runs
    noise_stats = df.groupby('run')['UV_1_280_mAU'].std().reindex(sorted_runs).
values

```

```

# Calculate SNR using peak detection (finding local maxima above a threshold)
# Threshold is set to 2x the noise std dev to identify significant peaks
threshold = 2 * np.max(noise_stats)
peak_heights = []

for run in sorted_runs:
    run_data = df[df['run'] == run]['UV_1_280_mAU'].values
    # Find local maxima above threshold
    peaks = []
    for i in range(1, len(run_data) - 1):
        if run_data[i] > run_data[i-1] and run_data[i] > run_data[i+1] and
            run_data[i] > threshold:
            peaks.append(run_data[i])
    if peaks:
        peak_heights.append(np.median(peaks))
    else:
        peak_heights.append(np.nan)

# Calculate Noise Variance (std**2) for plotting
noise_variance = noise_stats ** 2

# Create composite figure
fig, axes = plt.subplots(2, 1, figsize=(12, 8))

# Top subplot: Noise Variance vs Fraction_number
for i, run in enumerate(sorted_runs):
    run_data = df[df['run'] == run]
    run_fractions = run_data['Fraction_number'].values
    run_noise_var = noise_variance[i]
    axes[0].plot(run_fractions, run_noise_var, label=f'Run_{run}', marker='o',
                 markersize=5)

axes[0].set_xlabel('Fraction_number')
axes[0].set_ylabel('Noise_Variance')
axes[0].set_title('Noise_Variance_vs_Fraction_number')
axes[0].legend(loc='upper_right')
axes[0].grid(True, alpha=0.3)

# Bottom subplot: SNR boxplot grouped by run
snr_df = pd.DataFrame({'run': sorted_runs, 'SNR': peak_heights})
axes[1].boxplot(snr_df['SNR'], labels=snr_df['run'], patch_artist=True)
axes[1].set_xlabel('Run')
axes[1].set_ylabel('SNR')
axes[1].set_title('SNR_by_Run')
axes[1].grid(True, alpha=0.3, axis='y')

# Add global title
fig.suptitle('Intra-Run_Noise_Variance_and_Drift_Detection')

plt.tight_layout()
plt.savefig('/workspace/Intra-Run_Noise_Variance_and_Drift_Detection.png')
plt.close()
###

```

Listing 2: Code_2

```

###python
import pandas as pd

```

```

import numpy as np
import os

def Code_Updates():
    # Load the specified data file
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Validate column existence before proceeding
    required_columns = ['run', 'UV_1_280_mAU', 'volume_ml']
    for col in required_columns:
        if col not in df.columns:
            raise KeyError(f"Required column '{col}' not found in dataframe.")

    # Filter immediately after loading to reduce memory usage
    df = df[df['run'].notna() & df['UV_1_280_mAU'].notna() & df['volume_ml'].notna()
            ()]

    # Select the required variables directly from filtered df
    df_subset = df[['run', 'UV_1_280_mAU', 'volume_ml']]

    # Calculate variance per run
    df_subset['variance'] = df_subset.groupby('run')['UV_1_280_mAU'].var()

    # Calculate global mean and std for variance
    global_mean_var = df_subset['variance'].mean()
    global_std_var = df_subset['variance'].std()

    # Identify runs with variance exceeding mean + 2*std
    threshold = global_mean_var + 2 * global_std_var
    df_subset['is_outlier_run'] = df_subset['variance'] > threshold

    # Filter to only flagged runs
    flagged_runs = df_subset[df_subset['is_outlier_run']]

    # Count negative volume_ml outliers within flagged runs (value < -5 mL)
    flagged_runs['negative_volume_outlier'] = (flagged_runs['volume_ml'] < -5)
    flagged_runs['negative_outlier_count'] = flagged_runs['negative_volume_outlier']
        .sum()

    # Prepare output table (limit to top 10)
    output_table = flagged_runs[['run', 'variance', 'negative_outlier_count']].
        head(10)

    # Calculate total count of negative 'volume_ml' values across all flagged runs
    total_negative_outliers = flagged_runs['negative_outlier_count'].sum()

    # Create markdown output using vectorized string operations
    md_content = "# Outlier Run Identification Results\n\n"
    md_content += "## Flagged Outlier Runs\n\n"
    md_content += "| run | variance | negative_outlier_count |\n"
    md_content += "|-----|-----|-----|\n"
    runs_list = output_table['run'].astype(str).tolist()
    variances_list = output_table['variance'].astype(str).tolist()
    counts_list = output_table['negative_outlier_count'].astype(str).tolist()
    md_content += "|_|".join(r + " |" + v + " |" + c for r, v, c in zip(
        runs_list, variances_list, counts_list)) + "|\n"

    md_content += f"\n## Summary\n\n"

```

```
md_content += f"Total count of negative 'volume_ml' values across all flagged runs: {total_negative_outliers}*\n"

# Save to markdown file in /workspace/
output_path = '/workspace/outlier_run_identification.md'
with open(output_path, 'w') as f:
    f.write(md_content)

print(f"Output saved to {output_path}")
###
```

Listing 3: Code_3